

Details of Dependency Labeling Procedures

Algorithm 1 presents a heuristic for determining the dependency label between the headword of a constituent C and its corresponding governor in the constituency tree.

Algorithm 1: *assignDependencyLabel*

Input: Constituents C , c , P , and p , where C and P are the highest nodes that c and p are the heads of, respectively.

Output: The dependency label for the relation from p to c .

```

1: if  $C$ .label is UCP then
2:    $C \leftarrow C$ .left_most_child
3:  $ProcedureList \leftarrow [getSubjectLabel,$ 
4:    $getAdjunctLabel,$ 
5:    $getAdverbial*Label,$  ▷ for * in [Clause, Noun, Verb, Adjective, Preposition]
6:    $getFunctionalModifierLabel,$ 
7:    $getSinoLabel,$ 
8:    $getSoundLabel,$ 
9:    $getSubjectComplementLabel,$ 
10:   $get*ArgumentOfVerbLabel,$  ▷ for * in [Nominal, Prepositional, Adjectival, Verbal]
11:   $getClausalComplementOfVerbLabel,$ 
12:   $getComplementOfPrepositionLabel,$ 
13:   $getModifierOfNounLabel,$ 
14:   $getQuantifierLabel,$ 
15:   $getModifierOf*Label,$  ▷ for * in [ClassifierNoun, Adjective]
16:   $getClausalComplementOfAdjectiveLabel,$ 
17:   $getDiscourseConnectorLabel]$ 
18: for each  $procedure \in ProcedureList$  do
19:   if  $label \leftarrow procedure(C, P, c, p) \neq \text{none}$  then return  $label$ 
20: return  $dep$  ▷ return the unspecified dependency as default

```

The following procedures provide the exhaustive implementation of the Algorithm 1. Specifically, they formalize the deterministic rules used to derive dependency labels from the structural context of the constituency-to-dependency conversion. Together, these procedures ensure consistent label assignment across the full range of syntactic configurations handled by our framework.

Procedure $getSubjectLabel(C, P, c, p)$

```

if  $C$ .function_tag is SBJ then
  if  $C$ .phrase_tag in (NP, QNP) then return  $nsubj$ 
  if  $C$ .phrase_tag is ADJP then return  $asubj$ 
  if  $C$ .phrase_tag is VP then return  $vsubj$ 
  if  $C$ .phrase_tag in (S, SPL, SQ) then return  $csubj$ 
  if  $C$ .phrase_tag is PP then return  $psubj$ 
return  $none$ 

```

Procedure $getAdjunctLabel(C, P, c, p)$

```

if  $C$ .phrase_tag in (R, RP, QRP) then return  $adjunct$ 
return  $none$ 

```

Procedure $getAdverbialClauseLabel(C, P, c, p)$

```

if  $C$ .phrase_tag in (S, SPL, SQ, SBAR) then
  if  $C$ .phrase_tag is S then
    if  $hasTag(C, VP-MNR)$  or  $hasTag(C, ADJP-PRD-MNR)$  then return  $advcl$ 
  if  $C$ .function_tag in (PRP, CNC, MNR, ADV, CND) then return  $advcl$ 
  if  $C$ .function_tag is TMP then return  $c_tmod$ 
return  $none$ 

```

```

Procedure getAdverbialNounLabel(C, P, c, p)
  if C.phrase_tag in (NP | QNP) then
    if C.function_tag in (PRP, CNC, MNR, ADV, CND) then return n_advmod
    if C.function_tag is LOC then return n_lmod
    if C.function_tag is TMP then return n_tmod
  return none

```

```

Procedure getAdverbialVerbLabel(C, P, c, p)
  if C.phrase_tag is VP then
    if C.function_tag in (PRP, CNC, MNR, ADV, CND) then return v_advmod
    if C.function_tag is LOC then return v_lmod
    if C.function_tag is TMP then return v_tmod
  return none

```

```

Procedure getAdverbialAdjectiveLabel(C, P, c, p)
  if C.phrase_tag is ADJP then
    if C.function_tag is LOC then return a_lmod
    if C.function_tag is TMP then return a_tmod
  return none

```

```

Procedure getAdverbialPrepositionLabel(C, P, c, p)
  if C.phrase_tag in (PP, QPP) then
    if C.function_tag in (PRP, CNC, MNR, ADV, CND) then return p_advmod
    if C.function_tag is LOC then return p_lmod
    if C.function_tag is TMP then return p_tmod
  return none

```

```

Procedure getFunctionalModifierLabel(C, P, c, p)
  if C.function_tag in (PRN, TPC) then return parataxis
  if C.function_tag is VOC then return vocative
  if C.function_tag in (HLN, TTL) then return timod
  if C.pos_tag in (E, M) or C.function_tag in (MDP, CMD) then return intj
  return none

```

```

Procedure getSinoLabel(C, P, c, p)
  if C.phrase_tag contains _w then return sino
  return none

```

```

Procedure getSoundLabel(C, P, c, p)
  if C.pos_tag is ON then return sound
  return none

```

```

Procedure getSubjectComplementLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) and p.pos_tag is Vc then
    if C.phrase_tag in (NP, QNP, QP) then return n_sc
    if C.phrase_tag is ADJP then return a_sc
    if C.phrase_tag is VP then return v_sc
    if C.phrase_tag is PP then return p_sc
    if C.phrase_tag in (S, SPL, SQ) then return c_sc
  return none

```

```

Procedure getNominalArgumentOfVerbLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) and p.phrase_tag in (NP, QNP, QP) then
    if C.function_tag is DOB then return n_dobj
    if C.function_tag is IOB then return n_iobj
    if C.function_tag is CMP then return ncomp
    return nmod
  return none

```

```

Procedure getPrepositionalArgumentOfVerbLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) and p.phrase_tag is PP then
    if C.function_tag is IOB then return p_iobj
    if C.function_tag is CMP then return p_comp
    return p_mod
  return none

```

```

Procedure getAdjectivalArgumentOfVerbLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) and C.phrase_tag is ADJP then
    if C.function_tag is CMP then return a_comp
    return a_mod
  return none

```

```

Procedure getVerbalArgumentOfVerbLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) and C.phrase_tag is VP then
    if C.function_tag is CMP then return v_comp
    if c.pos_tag is D then return dir
    return v_mod
  return none

```

```

Procedure getClausalComplementOfVerbLabel(C, P, c, p)
  if P.phrase_tag in (VP, QVP) then
    if C.phrase_tag in (S, SQ, SBAR) then
      if hasSubject(C) then return c_comp
      if hasTag(C, VP) then return x_comp : v
      if hasTag(C, ADJP) then return x_comp : a
      if hasTag(C, NP) or hasTag(C, QNP) then return x_comp : n
      if hasTag(C, PP) then return x_comp : p
    return none

```

```

Procedure getComplementOfPrepositionLabel(C, P, c, p)
  if C.phrase_tag in (PP, QPP) then return p_c_comp
  return none

```

```

Procedure getModifierOfNounLabel(C, P, c, p)
  if P.phrase_tag in (NP, QNP) then
    if C.phrase_tag is QP or C.pos_tag is Num then return num
    if C.pos_tag in (Nw, Nq) then return quantifier
    if C.phrase_tag is QNP or C.pos_tag in (Pd, Pp) then return det
    if C.phrase_tag is NP or C.pos_tag in (Nr, Nt, Nu, Nun, Nn) then return n_mod
    if C.phrase_tag is ADJP or C.pos_tag in (Aa, An, VA, NA) then return a_mod
    if C.phrase_tag is VP or C.pos_tag in (Ve, Vc, Vcp, Vv, VN) then return v_mod
    if C.phrase_tag is PP then return p_mod
    if C.phrase_tag in (S | SBAR | SQ) then return r_c_mod
  return none

```

```

Procedure getQuantifierLabel(C, P, c, p)
  if P.phrase_tag is QP then
    if C.pos_tag is Num then return number
    if C.pos_tag is Nq then return quantifier
    return quant_mod
  return none

```

```

Procedure getModifierOfClassifierNounLabel(C, P, c, p)
  if C.pos_tag is Nn_swsp then
    if c.pos_tag is Ncs then return ncs
    if c.pos_tag is Nc then return nc
  return none

```

Procedure getModifierOfAdjectiveLabel(C, P, c, p)

```
if  $P.phrase\_tag$  is ADJP then
  if  $C.phrase\_tag$  in (NP, QNP, QP) or  $C.pos\_tag$  in (Nn, Pp) then return nmod
  if  $C.phrase\_tag$  is ADJP or  $C.pos\_tag$  in (Aa, An, VA, NA) then return amod
  if  $C.phrase\_tag$  is VP or  $C.pos\_tag$  in (Ve, Vc, Vcp, Vv, VN) then return vmod
  if  $C.phrase\_tag$  is PP, QPP then return pmod
return none
```

Procedure getClausalComplementOfAdjectiveLabel(C, P, c, p)

```
if  $P.phrase\_tag$  is ADJP then
  if  $C.phrase\_tag$  in (S, SQ, SBAR) then
    if hasSubject( $C$ ) then return ccomp
    if hasTag( $C, VP$ ) then return xcomp:v
    if hasTag( $C, ADJP$ ) then return xcomp:a
    if hasTag( $C, NP$ ) or hasTag( $C, QNP$ ) then return xcomp:n
    if hasTag( $C, PP$ ) then return xcomp:p
return none
```

Procedure getDiscourseConnectorLabel(C, P, c, p)

```
if  $C.phrase\_tag$  in (LBRK, RBRK) or  $C.pos\_tag$  is PU then return punct
if  $C.phrase\_tag$  is CONJP or  $C.pos\_tag$  is Cp then return cc
if  $C.pos\_tag$  is Cs then return mark
return none
```
